

JOBSHEET 9

STACK



Nama : Fadila Mayamasita

NIM : 254107020012

Kelas : 1C Teknik Informatika

Sarjana Terapan Teknik Informatika

Jurusan Teknologi Informasi

Politeknik Negeri Malang

1. Tujuan Praktikum

1. Membuat struktur data Stack
2. Menerapkan algoritma Stack ke dalam program Java

2. . Praktikum

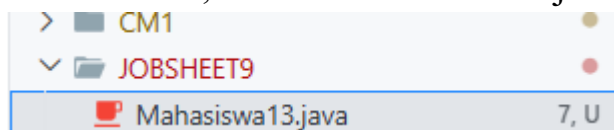
2.1 Percobaan 1: Mahasiswa Mengumpulkan Tugas

Sejumlah mahasiswa mengumpulkan berkas tugas di meja dosen secara ditumpuk dengan menerapkan prinsip stack. Dosen melakukan penilaian secara terurut mulai dari berkas tugas teratas. Selanjutnya, untuk mengumpulkan berkas tugas, diperlukan class StackTugasMahasiswa yang berperan sebagai Stack tempat menyimpan data tugas mahasiswa. Atribut dan method yang terdapat di dalam class StackTugasMahasiswa merepresentasikan pengolahan data menggunakan struktur Stack.

2 .1.1 Langkah-langkah Percobaan

A. Class Mahasiswa

1. Buat folder baru bernama Jobsheet9 di dalam repository Praktikum ASD. Buat file baru, beri nama Mahasiswa.java



2. Lengkapi class Mahasiswa dengan atribut yang telah digambarkan di dalam class diagram Mahasiswa, yang terdiri dari atribut nama, nim, kelas, dan nilai
3. Tambahkan konstruktor berparameter pada class Mahasiswa sesuai dengan class diagram Mahasiswa. Berikan nilai default nilai = -1 sebagai nilai awal ketika tugas belum dinilai

```
public class Mahasiswa13 {  
    String nama, nim, kelas;  
    int nilai;  
    public Mahasiswa13(String nama, String nim, String kelas) {  
        this.nama = nama;  
        this.nim = nim;  
        this.kelas = kelas;  
        nilai = -1;  
    }  
}
```

4. Tambahkan method tugasDinilai() yang digunakan untuk mengeset nilai ketika dilakukan penilaian tugas mahasiswa

```
void tugasDinilai(int nilai) {  
    this.nilai = nilai;  
}
```

B. Class StackTugasMahasiswa

5. Setelah membuat class Mahasiswa, selanjutnya perlu dibuat class StackTugasMahasiswa.java sebagai tempat untuk mengelola tumpukan tugas. Class StackTugasMahasiswa merupakan penerapan dari struktur data Stack

Mahasiswa13.java	1, U
StackTugasMahasiswa13.java	2, U

6. Lengkapi class StackTugasMahasiswa dengan atribut yang telah digambarkan di dalam class diagram StackTugasMahasiswa, yang terdiri dari atribut stack, size, dan top

```
public class StackTugasMahasiswa13 {  
    Mahasiswa13[] stack;  
    int top;  
    int size;
```

7. Tambahkan konstruktor berparameter pada class StackTugasMahasiswa untuk melakukan inisialisasi kapasitas maksimum data tugas mahasiswa yang dapat disimpan di dalam Stack, serta mengeset indeks awal dari pointer top

```
public StackTugasMahasiswa13(int size) {  
    this.size = size;  
    stack = new Mahasiswa13[size];  
    top = -1;  
}
```

8. Selanjutnya, buat method isFull bertipe boolean untuk mengecek apakah tumpukan tugas mahasiswa sudah terisi penuh sesuai kapasitas

```
public boolean isFull() {  
    if (top == size - 1) {  
        return true;  
    } else {  
        return false;  
    }  
}
```

9. Pada class StackTugasMahasiswa, buat method isEmpty bertipe boolean untuk mengecek apakah tumpukan tugas masih kosong

```
    }  
    public boolean isEmpty() {  
        if (top == -1) {  
            return true;  
        } else {  
            return false;  
        }  
    }
```

10. Untuk dapat menambahkan berkas tugas ke dalam tumpukan Stack, maka buat method push. Method ini menerima parameter mhs yang berupa object dari class Mahasiswa

```
    public void push(Mahasiswa13 mhs) {  
        if (!isFull()) {  
            top++;  
            stack[top] = mhs;  
        } else {  
            System.out.println(x: "Stack penuh! Tidak bisa menambahkan tugas lagi.");  
        }  
    }
```

11. Penilaian tugas mahasiswa yang dilakukan oleh dosen dilakukan dengan menggunakan method pop untuk mengeluarkan tugas yang akan dinilai. Method ini tidak menerima parameter apapun namun mempunyai nilai kembalian berupa object dari class Mahasiswa

```
    }  
    public Mahasiswa13 pop() {  
        if (!isEmpty()) {  
            Mahasiswa13 mhs = stack[top];  
            top--;  
            return mhs;  
        } else {  
            System.out.println(x: "Stack kosong! Tidak ada tugas untuk dinilai.");  
            return null;  
        }  
    }  
}
```

12. Buat method peek untuk dapat mengecek tumpukan tugas mahasiswa yang berada di posisi paling atas

```
    public Mahasiswa13 peek() {  
        if (!isEmpty()) {  
            return stack[top];  
        } else {  
            System.out.println(x: "Stack kosong! Tidak ada tugas yang di kumpulkan.");  
            return null;  
        }  
    }  
}
```

13. Tambahkan method print untuk dapat menampilkan semua daftar tugas mahasiswa pada Stack

```
public void print() {  
    for (int i = 0; i <= top; i++) {  
        System.out.println(stack[i].nama + "\t" + stack[i].nim + "\t" + stack[i].kelas );  
    }  
    System.out.println(x: "");  
}
```

C. Class StackTugasMahasiswa

14. Buat file baru, beri nama MahasiswaDemo.java

 MahasiswaDemo13.java 2, U

15. Tuliskan struktur dasar bahasa pemrograman Java yang terdiri dari fungsi main

```
public class MahasiswaDemo13 {  
    Run | Debug | Run main | Debug main  
    public static void main(String[] args) {  
  
    }  
}
```

16. Di dalam fungsi main, lakukan instansiasi object StackTugasMahasiswa bernama stack dengan nilai parameternya adalah 5.

```
Run | Debug | Run main | Debug main  
public static void main(String[] args) {  
    StackTugasMahasiswa13 stack = new StackTugasMahasiswa13(size: 5);  
}
```

17. Deklarasikan Scanner dengan nama variabel scan dan variabel pilih bertipe int

```
Scanner scanner = new Scanner(System.in);
```

18. Tambahkan menu untuk memfasilitasi pengguna dalam memilih operasi Stack dalam mengelola data tugas mahasiswa menggunakan struktur perulangan do-while.

```

public static void main(String[] args) {
    do {
        System.out.println(x: "\nMenu:");
        System.out.println(x: "1. Mengumpulkan Tugas");
        System.out.println(x: "2. Menilai Tugas");
        System.out.println(x: "3. Melihat Tugas Teratas");
        System.out.println(x: "4. Melihat Daftar Tugas");
        System.out.print(s: "Pilih: ");
        pilih = scan.nextInt();
        scan.nextLine();

        switch (pilih) {
            case 1: // Mengumpulkan Tugas (Push)
                System.out.print(s: "Nama: ");
                String nama = scan.nextLine();
                System.out.print(s: "NIM: ");
                String nim = scan.nextLine();
                System.out.print(s: "Kelas: ");
                String kelas = scan.nextLine();

                Mahasiswa13 mhs = new Mahasiswa13(nama, nim, kelas);
                stack.push(mhs);
                System.out.printf(format: "Tugas %s berhasil dikumpulkan\n", mhs.nama);
                break;

            case 2: // Menilai Tugas (Pop)
                Mahasiswa13 dinilai = stack.pop();
                if (dinilai != null) {
                    System.out.println("Menilai tugas dari " + dinilai.nama);
                    System.out.print(s: "Masukkan nilai (0-100): ");
                    int nilai = scan.nextInt();
                    dinilai.tugasDinilai(nilai);
                    System.out.printf(format: "Nilai Tugas %s adalah %d\n", dinilai.nama, nilai);
                }
                break;

            case 3: // Melihat Tugas Teratas (Peek)
                Mahasiswa13 lihat = stack.peek();
                if (lihat != null) {
                    System.out.println("Tugas terakhir dikumpulkan oleh " + lihat.nama);
                }
                break;

            case 4: // Melihat Daftar Tugas (Print)
                System.out.println(x: "Daftar semua tugas");
                System.out.println(x: "Nama\tNIM\tKelas");
                stack.print();
                break;

            default:
                System.out.println(x: "Pilihan tidak valid.");
        }
    } while (pilih >= 1 && pilih <= 4);
}

```

19. Commit dan push kode program ke Github

20. Compile dan run program.

2. 1.2 Verifikasi Hasil Percobaan

Cocokkan hasil compile kode program Anda dengan gambar berikut ini.

```
PS C:\Users\Fadila Mayamasita\OneDrive\Documents\ASD semester 2\Algoritma-struktur-data> & 'C:\F
a.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Fadila Mayamasita\AppData\Roamir
6a09dd7401beec7ed8963f5fb86eb\redhat.java\jdt_ws\Algoritma-struktur-data_c4fc32f9\bin' 'JOBSHEET9

Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 1
Nama: Dila
NIM: 1001
Kelas: 1A
Tugas Dila berhasil dikumpulkan

Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 1
Nama: Erik
NIM: 1002
Kelas: 1B
Tugas Erik berhasil dikumpulkan

Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 3
Tugas terakhir dikumpulkan oleh Erik

Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 1
Nama: Tika
NIM: 1003
Kelas: 1C
Tugas Tika berhasil dikumpulkan
```

```

Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 4
Daftar semua tugas
Nama    NIM    Kelas
Dila    1001    1A
Erik    1002    1B
Tika    1003    1C

```

```

Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 2
Menilai tugas dari Tika
Masukkan nilai (0-100): 87
Nilai Tugas Tika adalah 87

```

```

Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 4
Daftar semua tugas
Nama    NIM    Kelas
Dila    1001    1A
Erik    1002    1B

```

2. 1.3 Pertanyaan

1. Lakukan perbaikan pada kode program, sehingga keluaran yang dihasilkan sama dengan verifikasi hasil percobaan! Bagian mana yang perlu diperbaiki?

Jawaban:

- Urutan pencetakan pada method print() agar hasil sesuai dengan verifikasi. Daftar tugas harus ditampilkan dari yang paling baru dikumpulkan ke yang terlama. Oleh karena itu, perulangan pada method print() harus memakai decreementing loop for (int i = top; i >= 0; i--).

```

public void print() {
    //for (int i = 0; i <= top; i++)
    for (int i = top; i >= 0; i--) {
        System.out.println(stack[i].nama + "\t" + stack[i].nim + "\t" + stack[i].kelas );
    }
}

```

- Pembersihan buffer di class utama, setelah memanggil scanner.nextInt(), harus menambahkan scan.nextLine() agar input teks (kaya nama) pada menu berikutnya tidak terlewat.

```

pilih = scan.nextInt();
scan.nextLine();

scan.nextLine(); // buffer flush

```

- c. Urutan parameter konstruktor, memastikan instansi objek mahasiswa13 mengikuti urutan (nama, nim, kelas) atau sesuai dengan urutan yang didefinisikan pada class Mahasiswa13

2. Berapa banyak data tugas mahasiswa yang dapat ditampung di dalam Stack? Tunjukkan potongan kode programnya!

Jawaban:

Banyaknya data yang dapat ditampung ditentukan saat instansi objek stack class main, stack dapat menampung 5 data

Potongan kode program :

```
StackTugasMahasiswa13 stack = new StackTugasMahasiswa13(size: 5);
```

3. Mengapa perlu pengecekan kondisi !isFull() pada method push? Kalau kondisi if-else tersebut dihapus, apa dampaknya?

Jawaban:

Pengecekan isFULL() itu diperlukan buat mastiin masih ada ruang kosong dalam array sebelum nambahin data baru, dampaknya kalau dihapus ? jika stack telah penuh dan tetap coba menambah data maka akan terjadi error yang bikin program berhenti mendadak.

4. Modifikasi kode program pada class MahasiswaDemo dan StackTugasMahasiswa sehingga pengguna juga dapat melihat mahasiswa yang pertama kali mengumpulkan tugas melalui operasi lihat tugas terbawah!

Jawaban:

```
public void peekBottom() {  
    if (!isEmpty()) {  
        System.out.println("Tugas Pertama dikumpulkan oleh: " + stack[0].nama);  
    } else {  
        System.out.println(x: "Stack kosong!");  
    }  
}
```

Fungsinya buat melihat atau menampilkan data yang berada di posisi paling bawah dalam tumpukan tanpa menghapus data tersebut

5. Tambahkan method untuk dapat menghitung berapa banyak tugas yang sudah dikumpulkan saat ini, serta tambahkan operasi menunya!

Jawaban:

```
public int getCount() {  
    return top + 1;  
}
```

Fungsinya buat menghitung dan mengembalikan jumlah total tugas mahasiswa yang saat ini tersimpan di dalam tumpukan (*stack*)

6. Commit dan push kode program ke Github

2.2 Percobaan 2: Konversi Nilai Tugas Ke Binar

Waktu percobaan: 60 menit

Sampai tahap ini, proses pengelolaan data tugas mahasiswa menggunakan konsep Stack telah berhasil dibuat pada Percobaan 1. Selanjutnya, pada Percobaan 2 ini ditambahkan method baru yang berfungsi untuk mengonversi nilai tugas bertipe int ke dalam bentuk biner setelah tugas tersebut diberi nilai dan dikeluarkan dari Stack.

2.1 Langkah-langkah Percobaan

1. Buka kembali file StackTugasMahasiswa.java
2. Tambahkan method konversiDesimalKeBiner dengan menerima parameter kode bertipe int

```
public String konversiDesimalkeBiner(int nilai) {  
    StackKonversi13 stack = new StackKonversi13();  
    while (nilai > 0) {  
        int sisa = nilai % 2;  
        stack.push(sisa);  
        nilai = nilai / 2;  
    }  
    String biner = new String();  
    while (!stack.isEmpty()) {  
        biner += stack.pop();  
    }  
    return biner;  
}
```

Pada method ini, terdapat penggunaan StackKonversi yang merupakan penerapan Stack, sama halnya dengan class StackTugasMahasiswa. Hal ini bertujuan agar Stack untuk mahasiswa berbeda dengan Stack yang digunakan untuk biner karena tipe data yang digunakan berbeda. Oleh karena itu, buat file baru bernama StackKonversi.java

3. Tambahkan empat method yaitu isEmpty, isFull, push, dan pull sebagai operasi utama Stack pada class StackKonversi

```

4
3 public class StackKonversi13 {
4     int[] tumpukanBiner;
5     int size;
6     int top;
7
8     public StackKonversi13() {
9         this.size = 32;
10        tumpukanBiner = new int[size];
11        top = -1;
12    }
13
14    public boolean isEmpty() {
15        return top == -1;
16    }
17
18    public boolean isFull() {
19        return top == size - 1;
20    }
21
22    public void push(int data) {
23        if (isFull()) {
24            System.out.println(x: "Stack penuh");
25        } else {
26            top++;
27            tumpukanBiner[top] = data;
28        }
29    }
30
31    public int pop() {
32        if (isEmpty()){
33            System.out.println(x: "Stack kosong");
34            return -1;
35        } else {
36            int data = tumpukanBiner[top];
37            top--;
38            return data;
39        }
40    }
41 }

```

4. Agar nilai tugas mahasiswa dikonversi ke dalam bentuk biner setelah dilakukan penilaian, maka tambahkan baris kode program pada method pop di class MahasiswaDemo

```

case 2: // Menilai Tugas (Pop)
Mahasiswa13 dinilai = stack.pop();
if (dinilai != null) {
    System.out.println("Menilai tugas dari " + dinilai.nama);
    System.out.print(s: "Masukkan nilai (0-100): ");
    int nilai = scan.nextInt();
    dinilai.tugasDinilai(nilai);
    System.out.printf(format: "Nilai Tugas %s adalah %d\n", dinilai.nama, nilai);
    String biner = stack.konversiDesimalkeBiner(nilai);
    System.out.println("Nilai Biner Tugas: " + biner);
}

```

5. Compile dan run program.
6. Commit dan push kode program ke Github

2.2.2 Verifikasi Hasil Percobaan

Cocokkan hasil compile kode program Anda dengan gambar berikut ini.

```
Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 1

Nama: Tika
NIM: 1003
Kelas: 1C
Tugas Tika berhasil dikumpulkan

Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 2

Menilai tugas dari Tika
Masukkan nilai (0-100): 87
Nilai Tugas Tika adalah 87
Nilai Biner Tugas: 1010111
```

2. 2.3 Pertanyaan

1. Jelaskan alur kerja dari method konversiDesimalKeBiner!

Jawaban:

Alur kerja pada konversiDesimalkeBiner menggunakan stack (LIFO) untuk menyimpan sisa pembagian dan menampilkannya di urutan yang benar.

- a. Iterasi pembagian : selama nilai desimalnya lebih besar dari 0, pembagiannya bakal terus – menerus menggunakan angka 2.
- b. Penyimpanan sisa : sisa hasil bagi (nilai % 2) dimasukin ke dalam stackkonversi, sia ini merupakan angka biner yang didapat dari urutan belakang.

- c. Pembaruan nilai : nilai desimal diperbarui dengan hasil bagi nilai / 2 buat iterasi berikutnya.
 - d. Pengambilan data (pop) : setelah nilai desimal 0, semua angka biner yang tersimpan di dalam stack diambil satu per satu.
 - e. Pembentukan string : karena pakai stack, angka terakhir masuk (sisa pembagian terakhir) bakal keluar pertama kali, sehingga susunan angka biner menjadi benar.
2. Pada method konversiDesimalKeBiner, ubah kondisi perulangan menjadi while (kode != 0), bagaimana hasilnya? Jelaskan alasannya!

Jawaban:

```
Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 2

Menilai tugas dari Tika
Masukkan nilai (0-100): 87
Nilai Tugas Tika adalah 87
Nilai Biner Tugas: 1010111
```

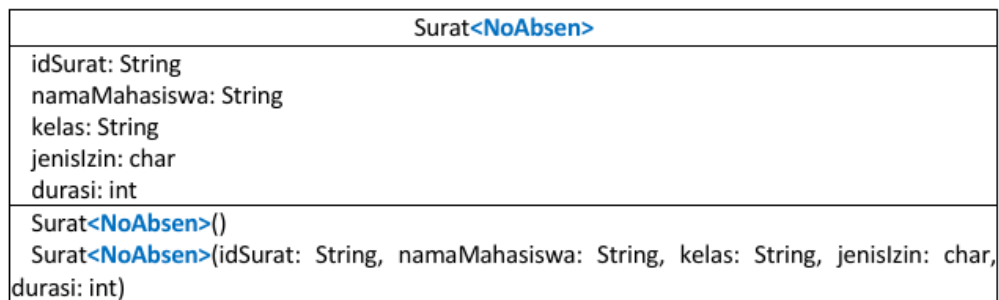
Hasil konversinya akan tetap sama dan program berjalan normal buat bilangan positif. Alasannya karena pembagian nilai bulat positif buat cari biner, kondisi nilai > 0 dan nilai != 0 itu punya titik henti yang sama yaitu ketika nilai sudah mencapai angka 0. Namun penggunaan nilai != 0 lebih fleksibel jika program dikembangkan buat menangani bilangan negatif.

2.4 Latihan Praktikum

Waktu : 90 Menit

Mahasiswa mengajukan surat izin (karena sakit atau keperluan lain) setiap kali tidak mengikuti perkuliahan. Surat terakhir yang masuk akan diproses atau divalidasi lebih dulu oleh admin Prodi. Perhatikan class diagram berikut.

Perhatikan class diagram berikut.



Atribut jenisIzin digunakan untuk menyimpan keterangan ijin mahasiswa (S: sakit atau I: izin keperluan lain) dan durasi untuk menyimpan lama waktu izin.

Berdasarkan class diagram tersebut, implementasikan class Surat dan tambahkan class StackSurat untuk mengelola data Surat. Pada class yang memuat method main, buat pilihan menu berikut:

1. Terima Surat Izin untuk memasukkan data surat
2. Proses Surat Izin untuk memproses atau memverifikasi surat
3. Lihat Surat Izin Terakhir untuk melihat surat teratas
4. Cari Surat untuk mencari ada atau tidaknya surat izin berdasarkan nama mahasiswa

Jawaban:

a. Membuat Class Surat

Membuat objeknya yaitu surat, tiap surat punya identitas lengkap

```
public class Surat13 {  
    String idSurat, namaMahasiswa, kelas;  
    char jenisIzin; // 'S' untuk sakit, 'I' untuk izin  
    int durasi;  
  
    public Surat13(String idSurat, String namaMahasiswa, String kelas, char jenisIzin, int durasi) {  
        this.idSurat = idSurat;  
        this.namaMahasiswa = namaMahasiswa;  
        this.kelas = kelas;  
        this.jenisIzin = jenisIzin;  
        this.durasi = durasi;  
    }  
}
```

b. Membuat Class StackSurat

buat pengelolanya, membutuhkan array untuk menyimpan surat variabel size
buat kapasitas kotak dan top sebagai penanda posisi surat paling atas

```
JOBSHEET 19 / StackSurat13.java / Language Support for Java (RMJ) by Red Hat / StackSurat13 / peek()

1  package JOBSHEET9;
2
3  public class StackSurat13 {
4      Surat13[] tumpukan;
5      int size, top;
6
7  public StackSurat13(int size) {
8      this.size = size;
9      this.tumpukan = new Surat13[size];
10     this.top = -1; // -1 tumpukan masih kosong
11 }
12
13 // pengecekan apakah kotak penuh
14 public boolean isFull() {
15     return top == size - 1;
16 }
17
18 // pengecekan apakah kotak kosong
19 public boolean isEmpty() {
20     return top == -1;
21 }
22
23 // 1. menerima surat (push)
24 public void push(Surat13 s) {
25     if (!isFull()) {
26         top++;
27         tumpukan[top] = s;
28     } else {
29         System.out.println(x: "Kotak Penuh! tidak bisa menerima surat lagi.");
30     }
31 }
32
33 // 2. proses Surat (pop)
34 public Surat13 pop() {
35     if (!isEmpty()) {
36         Surat13 s = tumpukan[top];
37         top--;
38         return s;
39     }
40     return null;
41 }
42
43 // 3. melihat surat terakhir (peek)
44 public Surat13 peek() {
45     if (!isEmpty()) {
46         return tumpukan[top];
47     }
48     return null;
49 }
50
51 // 4. mencari surat berdasarkan nama (Linear Search)
52 public void cariSurat13(String nama) {
53     boolean ketemu = false;
54     for (int i = top; i >= 0; i--) {
55         if (tumpukan[i].namaMahasiswa.equalsIgnoreCase(nama)) {
56             System.out.println("Surat ditemukan pada posisi ke-" + (top - i + 1) + "dari atas.");
57             System.out.println("ID: " + tumpukan[i].idSurat + " | Alasan: " + tumpukan[i].jenisIzin);
58             ketemu = true;
59             break;
60         }
61     }
62     if (!ketemu) System.out.println("Surat atas nama" + nama + "tidak ditemukan.");
63 }
64 }
```

c. Menjalankan Sistem (Class Main) buat menu interaktifny

```
JOBSHEET9 > AdministrasiSurat13.java > ...
1 package JOBSHEET9;
2
3 import java.util.Scanner;
4 public class AdministrasiSurat13 {
    Run | Debug | Run main | Debug main
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7         StackSurat13 kotakSurat = new StackSurat13(size: 10);
8         int pilih;
9
10        do {
11            System.out.println(x: "\n-- Menu Admin Prodi ---");
12            System.out.println(x: "1. Terima Surat Izin");
13            System.out.println(x: "2. Proses Surat (Validasi)");
14            System.out.println(x: "3. Lihat Surat Terakhir");
15            System.out.println(x: "4. Cari Surat Berdasarkan Nama");
16            System.out.print(s: "Pilih menu: ");
17            pilih = sc.nextInt();
18            sc.nextLine(); // Pembersihan buffer
19
20            switch (pilih) {
21                case 1:
22                    System.out.print(s: "ID Surat: "); String id = sc.nextLine();
23                    System.out.print(s: "Nama: "); String nama = sc.nextLine();
24                    System.out.print(s: "Kelas: "); String kls = sc.nextLine();
25                    System.out.print(s: "Jenis (S/I): "); char jns = sc.next().charAt(index: 0);
26                    System.out.print(s: "Durasi (hari): "); int dur = sc.nextInt();
27                    kotakSurat.push(new Surat13(id, nama, kls, jns, dur));
28                    break;
29                case 2:
30                    Surat13 s = kotakSurat.pop();
31                    if (s != null) System.out.println("Memproses surat milik: " + s.namaMahasiswa);
32                    else System.out.println(x: "Tidak ada surat untuk diproses.");
33                    break;
34                case 3:
35                    Surat13 top = kotakSurat.peek();
36                    if (top != null) System.out.println("Surat teratas: " + top.namaMahasiswa);
37                    break;
38                case 4:
39                    System.out.print(s: "Masukkan nama yang dicari: ");
40                    String cari = sc.nextLine();
41                    kotakSurat.cariSurat13(cari);
42                    break;
43            }
44        } while (pilih >= 1 && pilih <= 4);
45    }
46 }
```

d. Menampilkan outputnya

```
--- Menu Admin Prodi ---
1. Terima Surat Izin
2. Proses Surat (Validasi)
3. Lihat Surat Terakhir
4. Cari Surat Berdasarkan Nama
Pilih menu: 1
ID Surat: S001
Nama: James
Kelas: 1A
Jenis (S/I): S
Durasi (hari): 3

--- Menu Admin Prodi ---
1. Terima Surat Izin
2. Proses Surat (Validasi)
3. Lihat Surat Terakhir
4. Cari Surat Berdasarkan Nama
Pilih menu: 1
ID Surat: E002
Nama: Felix
Kelas: 1B
Jenis (S/I): I
Durasi (hari): 1

--- Menu Admin Prodi ---
1. Terima Surat Izin
2. Proses Surat (Validasi)
3. Lihat Surat Terakhir
4. Cari Surat Berdasarkan Nama
Pilih menu: 3
Surat teratas: Felix

--- Menu Admin Prodi ---
1. Terima Surat Izin
2. Proses Surat (Validasi)
3. Lihat Surat Terakhir
4. Cari Surat Berdasarkan Nama
Pilih menu: 4
Masukkan nama yang dicari: Felix
Surat ditemukan pada posisi ke-1dari atas.
```

```
--- Menu Admin Prodi ---
1. Terima Surat Izin
2. Proses Surat (Validasi)
3. Lihat Surat Terakhir
4. Cari Surat Berdasarkan Nama
Pilih menu: 2
Memproses surat milik: Felix

--- Menu Admin Prodi ---
1. Terima Surat Izin
2. Proses Surat (Validasi)
3. Lihat Surat Terakhir
4. Cari Surat Berdasarkan Nama
Pilih menu: 2
Memproses surat milik: James
```